

MML_DAY4

语言模型发展史

概念：语言模型的功能：输入字符串预测下一个字的概率，判断一个字符串成句概率。

N-gram模型

- 计算过程：
 - 输入语料库，例如：{今天天气好}，...
 - 统计每一个词出现在其它词组合的后的概率 $P(W_3|W_1W_2)$ ，例如： $P(\text{天}|\text{今天}) = 0.5$
 - 因为词语长的时候组合太多，所以一般使用马尔科夫假设：只看前N个词的组合，统计N+1出现的概率。
 - 成句概率计算：将每次的预测结果相乘，例如：
$$P(W_1W_2W_3) = P(W_1) * P(W_2|W_1) * P(W_3|W_1W_2)$$
 - 因为有时候会出现没有见过的组合，这个时候预测结果为0，为避免这种情况，会使用拉普拉斯平滑：计算概率时，为每个词计数+1，
$$P(w_i|ctx) = (C(ctx, w_i) + 1) / (C(ctx) + V)$$
 - C ：该情况出现的次数
 - V ：语料库词数量
- 模型特点
 - 推理速度极快：本质是查表+乘法
 - 数据稀疏：训练集难以覆盖所有词组合，导致未见过组合概率为0，虽然可以平滑解决，但是付出一定代价
 - 长距离依赖失效：长语句中，使用马尔科夫假设，视野大小为N，有时缺少必须要的推断信息
 - 语义泛化能力弱：仅仅统计词语组合概率，无法理解词语之间的联系，例如：猫和狗
 - 存储开销大：词语组合多，根据马尔科夫视野N，呈指数级变化

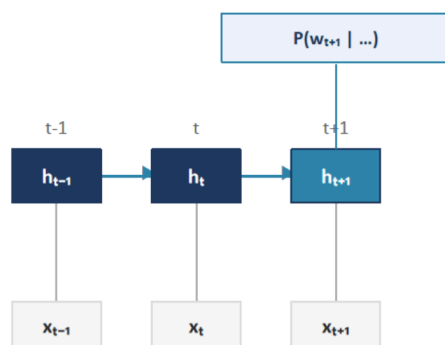
神经网络语言模型

- 计算过程：
 - 建立一个网络模型，一般是：Embedding+RNN+LSTM+Linear等等组合

- 输入：语料库中的字符串
- 输出：预测下一个字的概率

○ 模型训练：

- 输入： $x=\{0,N\}$ 字符串, $y=\{1, N+1\}$ 字符串
- 损失函数计算：因为网络中一般使用RNN等类似算子，将中间每一步迭代的结果和下一个字符做交叉熵，然后求和。
- 输出：softmax后，输出大小 V 的概率表，表示每个词是下一个词表的概率，**本质类似一个多分类模型**



○ 使用模型：

- 输入一个字符串，例如：{今天天气好}
- 计算每一个词的概率，例如： $P(\text{天}|\text{今})$, $P(\text{天}|\text{今天})$, $P(\text{气}|\text{今天天})$...
- 相乘得到这个句子的成句概率

- 计算困惑度：
$$\text{PPL} = \exp \left(-\frac{1}{N} \sum_{i=1}^N \log P(w_i | w_1, \dots, w_{i-1}) \right)$$
，因为多次相乘概率过低不利于比较

- 困惑度越低，表示成句概率越高
- 困惑度只有在：同一模型，同一长度比较时才有意义

● 模型特点

○ 推理速度：低于N-gram

○ 对比N-gram优势

- 数据稀疏：softmax天然平滑不会出现概率0
- 长距离依赖失效：一定程度解决，但是效果有限，受限RNN，LSTM
- 语义泛化能力：一定程度解决，Embedding能将相近的词语编码成类似的向量
- 存储开销：取决于网络参数，参数大小固定

BERT模型

- 核心理念
 - 预训练：为了防止针对每一个任务，都重头训练一个新的模型，训练一个通用的模型，根据具体的任务，再进行微调，减少重复训练的工作量

预训练 + 微调范式



- 双向上下文：同时利用左侧和右侧的词语信息，编码时使用mark代替中间的词，让模型去预测
- 规模优势：在当时模型的参数和训练数据都是巨大的，大概0.1B参数
- 预训练的任务
 - 任务一：完形填空任务，让模型预测中间值，同时利用前后文信息
 - 任务二：前后句关联，判断前后两个句子是否是有关联。事实证明，这个任务不重要。

第三部分-预训练语言模型

预训练任务一：Masked Language Model

任务设计

随机遮盖输入序列中 15% 的 Token，让模型预测被遮盖的原始词：

输入：我 喜欢 [MASK] 苹果
目标：预测 [MASK] = 「吃」

遮盖策略（15% Token 中）

- 80% 替换为 [MASK]
- 10% 替换为随机词
- 10% 保持原词不变

避免模型「只认识 [MASK]」，增强泛化能力

双向 vs 单向：为何 BERT 更好？

GPT（单向 LM）：逐词从左预测

我 喜欢 [MASK] 苹果
→→→（只看左侧）

BERT（双向 MLM）：利用完整上下文

我 喜欢 [MASK] 苹果
← 同时关注左右两侧上下文 →

双向上下文的优势

多义词消歧：「银行」在不同语境下有不同向量
长距离依赖：主语和谓语无论距离多远均可关联
丰富语义：综合前后词意获得更准确表示

第三部分-预训练语言模型

预训练任务二：Next Sentence Prediction

任务设计

给定两个句子，判断 B 是否是 A 的下一句。
训练时 50% 正例（真实相邻句），50% 负例：

IsNext（正例）
A：今天天气很好
B：我们去公园散步吧

NotNext（负例，随机替换）
A：今天天气很好
B：量子力学起源于20世纪初

NSP 帮助模型理解句子间逻辑关系

NSP 支持的下游任务

- 问答（QA）：问题与候选段落的匹配
- 自然语言推理：前提与假设是否蕴含

BERT 输入格式

将两句拼接后送入模型，用特殊 Token 分隔：

[CLS] Sentence_A [SEP] Sentence_B [SEP]

特殊 Token 的作用

- [CLS] 句对分类的汇聚表示，NSP 预测由它输出
- [SEP] 句子边界分隔符，区分 Sentence A 与 B

BERT 同时优化两个预训练目标：

Loss = MLM Loss + NSP Loss

- 模型结构
 - 输入：{seq, 768}
 - Embedding层：如下所示，同时对token，token位置，前后句关系都进行了编码，将其相加：{seq,768}
 - token编码：和普通的Embedding一样，每个token编码768，词表大小V，编码后{seq,768}
 - 位置编码：每个位置编码{768}向量，最大支持512个位置，编码后{512,768}，通过处理{seq,768}(只保留前seq个)
 - 前后句编码：每个独立句子编码{768}向量，最大支持2句，编码后{2,768}，扩展后{seq,768}(根据词的所在的句子进行扩展)

- 最后输出： $\{seq, 768\} = \{seq, 768\}(token) + \{seq, 768\}(pos) + \{seq, 768\}(NSP)$

模型架构



输出用于不同下游任务（分类、问答、NER...）

三种 Embedding 相加

Token Embedding

将每个 WordPiece Token 映射到稠密向量（词表约 30K）

Position Embedding

可学习的位置编码（非 sin/cos），表示 Token 在序列中的位置

Segment Embedding

区分句子 A（EA）和句子 B（EB），用于 NSP 任务

$$e = e_token + e_position + e_segment$$

输入示意（5 个 Token）

[CLS]	我	爱	[SEP]	NLP
v1	v2	v3	v4	v5
p0	p1	p2	p3	p4
sA	sA	sA	sA	sB

⊗ （逐元素相加）

最终 Embedding（送入 Transformer Block）

[CLS]: 汇聚整个句子的语义，用于分类任务
[SEP]: 标记句子边界，区分句子 A / B

- transform层：堆叠了12层transform层：输入/输出维度是一致的所以可以堆叠
- 输出： $\{seq, 768\}$ ，表示每个token对应上下文的向量特征，还需要接其它网络才能真正使用
- BERT模型不适配生成问题的原因
 - 训练目标不适配：目标是完形填空，目的是生成，目的和目标不一致
 - 输出长度固定：BERT是一次性推理，非自回归式推理，输出是固定维度的向量

Transform算子介绍

- 计算过程：

单个 Block 结构（自底向上）



残差连接（Residual Connection）

每个子层（MHA、FFN）的输出与输入相加再归一化：

$$\text{Output} = \text{LayerNorm}(x + \text{Sublayer}(x))$$

避免梯度消失：允许堆叠 12/24 层
恒等映射：最坏情况学习恒等函数

层归一化（Layer Normalization）

对每个位置的激活值独立归一化，
稳定训练过程，使深层网络收敛更快：

$$\text{LN}(x) = (x - \mu) / \sigma \cdot \gamma + \beta$$

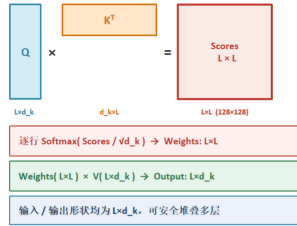
- 输入： $x = \{L, d_model\}$ 设 $\{128, 768\}$ ，L：token 长度， d_model token 编码维度
- 多头自注意力机制(核心):
 - 输入： $\{L, d_model\} = \{128, 768\}$
 - 首先通过三个全连接层 $\{d_model, d_model\}$ ，得到 QKV 矩阵： $\{L, d_model\} * 3 = \{128, 768\} * 3$
 - 设多头数量为 $h = 12$ ，将 QKV 矩阵划分为 12 份， $\{L, d_k\} * 3 * h = \{128 * 64\} * 3 * 12$
 - 将每一份矩阵做 $Q * K^T$ ，得到矩阵 S： $\{L, L\} * h = \{128 * 128\} * 12$
 - 对矩阵 S 逐行做 $\text{softmax}(S / \sqrt{d_k})$ 得到 A
 - 对每一份 A 做： $A * V = \{L, L\} * \{L, d_k\}$ ，得到： $\{L, d_k\} * h = \{128, 64\} * 12$
 - 拼接多头计算结果： $\{L, d_k\} * h = \{L, d_model\} = \{128, 768\}$
 - 全连接层， $\{L, d_model\} * \{d_model, d_model\} = \{L, d_model\} = \{128, 768\}$

- 输出：{L,d_model}={128,768}

逐步维度变化 (BERT-base 示例, L=128)

① 输入序列 X	
shape: $L \times d_{\text{model}}$ (128 × 768)	每个 Token 用 768 维向量表示
② 线性投影 $\rightarrow Q / K / V$	
shape: $L \times d_k$ (128 × 64)	各乘权重矩阵 $W_Q / W_K / W_V$
$d_k = d_{\text{model}} / h = 768 / 12 = 64$	
③ $Q \times K^T = \text{注意力分数矩阵}$	
Scores shape: $L \times L$ (128 × 128)	
$(L \times d_k) \times (d_k \times L) = L \times L$	
每行 i = Token i 对所有 Token 的相关性	
逐行 Softmax $\rightarrow$ 每行之和 = 1 (权重分布)	
④ 加权汇聚输出	
Output shape: $L \times d_k$ (128 × 64)	
Weights ($L \times L$) $\times V$ ($L \times d_k$) = $L \times d_k$	

矩阵形状可视化 (比例示意)



为什么是 $L \times L$?

矩阵含义
 Scores[i][j] = Token i 对 Token j 的相关性得分
 L 行 $\times L$ 列 = 序列中任意两个 Token 之间都参与一次相关性计算 (全局注意力)

逐行 Softmax 的含义
 行 i 的 Softmax 输出
 Weights[i] = softmax (Scores[i]) $\rightarrow$ 长度为 L 的概率向量
 该行之和 = 1.0
 Weights[i][j] 表示 Token i 从 Token j 借取信息的比例 (注意力权重)

Weights[i] = softmax (Q[i] $\cdot$ K^T / d_k)

注意力矩阵示例 (L=5, 每行之和 = 1.0)

Q $\downarrow$ K $\rightarrow$	[CLS]	我	爱	NLP	[SEP]	1
[CLS]	0.50	0.10	0.08	0.12	0.20	1.0
我	0.08	0.52	0.25	0.10	0.05	1.0
爱	0.06	0.30	0.30	0.28	0.06	1.0
NLP	0.08	0.22	0.32	0.30	0.08	1.0
[SEP]	0.22	0.10	0.08	0.10	0.50	1.0

权重强度: ≥ 0.40 ≥ 0.25 ≥ 0.10 < 0.10

○ 残差1计算:

- 输入: $x = \{L, d_{\text{model}}\}$ (原输入), Sublayer(x) = {L,d_model}(多头输出)={128,768}
- 计算: $output = LayerNorm(x + Sublayer(x))$, 对 $x = \{L, : \}$ 进行归一化
- 输出: {L,d_model}={128,768}

○ 前馈网络

- 输入: {L,d_model}={128,768}
- 全连接层1: {L,d_model}*{d_model,N}={L,N}={128,3072}
 - N经验参数: BERT取3072
- 激活函数: gelu
- 全连接层2: {L,N}*{N,d_model}={128,768}
- 输出: {L,d_model}={128,768}

○ 残差2计算: 和1一样

● 含义解释:

○ 多头自注意力机制(核心):

- 多头的含义: 表示提取不同的词关系特征, 比如: 主谓宾, 指代关系等。头相当于 channel。
- $Q * K^T$: 计算"当前词应该关注哪些词"的权重, 如下所示。
- $Q * K^T * V$: V 表示每个词携带的内容信息; 用这些权重对 V 加权求和后, 得到的就是当前词在这个注意力头下聚合到的内容特征
- 例如:
 - 假设当前头关注的是**主谓关系**
 - $softmax(\frac{Q * K^T}{\sqrt{d_k}})$: 计算 我 和其它词的关联度 $A = \{0.08, 0.25, \mathbf{0.52}, 0.1, 0.05\}$, 我 和 爱 是主谓关系所以权重最高

- $A * V : \{1,5\} * \{5,d_model\} \rightarrow N = \{1,d_model\}$, V表示词内容, 因为 爱 权重最高所以N从 爱 中获取的内容占比会最多。
- N表示当前句子中, 主谓关系中, 我 这个词的上下文表示。
- 对矩阵S逐行 $softmax(S/\sqrt{d_k})$: softmax是将权重归一化; $\sqrt{d_k} > 1$ 经验值, 为了使权重分布更均衡

- 残差计算: 梯度可以直接沿着残差(原值+修正)路径传播, 减轻梯度消失, 多层网络中常用
- 前馈网络: 增加可训练参数

Q↓ K→	[CLS]	我	爱	NLP	[SEP]	Σ
[CLS]	0.50	0.10	0.08	0.12	0.20	1.0
我	0.08	0.25	0.52	0.10	0.05	1.0
爱	0.06	0.30	0.30	0.28	0.06	1.0
NLP	0.08	0.22	0.32	0.30	0.08	1.0
[SEP]	0.22	0.10	0.08	0.10	0.50	1.0

权重强度: ≥ 0.40 ≥ 0.25 ≥ 0.10 < 0.10

